Expert Systems With Applications 299 (2026) 130220



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Fusing global attention and self-attention for better performance and interpretability on tabular data

Hung Bui [a,b], Bac Le [a,b,*]

[a] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
[b] Vietnam National University, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:
Tabular data
Global attention
Cross-attention
Interpretable artificial intelligence
Fusion attention neural network

ABSTRACT

Deep learning has demonstrated remarkable success across domains such as image, audio, video, and text processing. However, its application to tabular data remains challenging due to the heterogeneous nature of features and the lack of positional information, which complicates feature interaction modeling. While deep learning approaches for tabular data have achieved performance levels comparable to traditional methods like gradient boosting machines, their adoption in real-world scenarios is limited by issues of interpretability and robustness. We introduce FusANet (**Fus**ion **A**ttention Neural **Net**work), a novel parametric transformer-style architecture for tabular data that fuses a dataset-level global attention map with instance-level local self-attention using a cross-attention fusion module. FusANet (i) learns a shared, pre-initialized global attention matrix W_{global} to capture population-wide feature relationships, (ii) retains sample-specific interactions via local multi-head self-attention, and (iii) fuses these views with cross-attention to produce robust predictions and dual-level explanations. Across 10 benchmark tabular datasets, we show FusANet achieves average improvements up to 19.72 % in classification accuracy and up to 24.25 % relative RMSE reduction on regression task versus strong parametric state-of-the-art models and performs competitively with gradient-boosted trees, while offering dataset- and instance-level attributions.

1. Introduction

Deep learning has achieved remarkable success in fields such as natural language processing and computer vision, where data is typically homogeneous and well-structured. However, for tabular data, its heterogeneous nature poses unique challenges that make the benefits of deep learning less clear. Despite the potential of deep learning for tabular data, extensive research has been conducted to address its unique properties, resulting in various strategies for effective handling (Borisov et al., 2022; Gorishniy et al., 2021; Grinsztajn et al., 2022).

Tabular data often consists of a mix of heterogeneous features, such as continuous, categorical, and ordinal variables, each drawn from different distributions. This diversity complicates the ability of neural networks to model feature interactions effectively. Furthermore, since the order of columns in tabular data is arbitrary, models must be *permutation-equivariant*, meaning they must produce consistent outputs regardless of feature order. As Gorishniy et al. (2021) suggests, models for tabular data must be capable of learning irregular target functions, be robust to uninformative features, and capture complex interactions between features, emphasizing the need for efficient feature representations. Attention mechanisms, originally introduced in Transformers (Vaswani et al., 2017), have shown their effectiveness in these contexts (Grinsztajn et al., 2022). Applications include attention within individual samples to model feature interactions (Gorishniy et al., 2021; Song et al., 2018) and attention between samples in a dataset (Somepalli et al., 2021). Multi-scaled attention is also applied in various domains, but the effective integration of multi-scale (local and global) attention in tabular data remains underexplored.

In this paper, we propose a new fully parametric model for tabular data called FusANet. FusANet introduces a novel global attention mechanism to capture shared relationships between features across the entire dataset and combines this with a local attention mechanism, which is equivalent to the self-attention mechanism (Vaswani et al., 2017) to model feature interactions within individual samples. To achieve superior performance, we further propose a cross-attention mechanism to fuse the outputs of local and global attention layers. Unlike traditional Transformer architectures, which rely on positional embeddings, FusANet represents each sample as a set of token embeddings for its features, and handles numerical and categorical features differently during the embedding process to better capture their unique

* Corresponding author.
E-mail addresses: bqhung201@clc.fitus.edu.vn (H. Bui), lhbac@fit.hcmus.edu.vn (B. Le).

https://doi.org/10.1016/j.eswa.2025.130220
Received 7 June 2025; Received in revised form 19 October 2025; Accepted 27 October 2025
Available online 4 November 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

characteristics. The combined outputs from the attention layers are then used to generate the final context representations for downstream prediction tasks. It is important to clarify that the term *global attention* in our work refers specifically to an attention mechanism that computes a shared, learnable attention map across all features, applied uniformly to all instances in the dataset. This is in contrast to the more common use of "global attention" in the literature-such as in Natural Language Processing or Computer Vision transformers-where it typically denotes *instance-specific token-to-token attention computed over the entire input sequence or spatial layout*. Our global attention stands in contrast to *local attention*, which is computed independently for each instance (and possibly per feature group), thereby capturing interactions that are more specific to individual samples. In sum, FusANet introduces a learnable, shared global attention that captures dataset-level relations and integrates it with local self-attention through cross-attention, producing complementary signals for prediction and explanation. The architecture is set-based (order-free), field-aware (separate numeric/categorical encoders), and mixing-aware (no feature mixing before attention), which together constitute a new fusion-of-attentions paradigm for tabular learning.

To evaluate FusANet, we conduct extensive experiments across multiple datasets and tasks. Our benchmarking results show that FusANet consistently outperforms state-of-the-art deep learning models for tabular data: We achieve average improvements of 0.1 - 19.72 % in classification accuracy and 2.15–24.25 % relative RMSE reduction on regressions. Furthermore, FusANet provides enhanced interpretability: local attention weights reveal instance-specific feature importance, while global attention weights highlight the overall contribution of features across the dataset. By summing attention weights across multiple heads, we provide insights into how FusANet prioritizes features for individual predictions and how these features contribute to the overall trained model.

In summary, the main contributions of this paper are concluded as follows:

- We propose a novel **global attention mechanism** for tabular data, enabling the model to capture shared relationships between features across the entire dataset.
- We introduce **FusANet**, which combines global and local attention mechanisms and fuses their outputs through cross-attention for improved performance.
- We conduct extensive experiments, comparing FusANet with state-of-the-art deep learning models for tabular data across diverse datasets and tasks.
- We demonstrate the enhanced interpretability of FusANet in terms of global attention weights providing dataset-level insights.

2. Related work

2.1. Deep learning for tabular data

Deep learning models have shown great promise in handling tabular data, with various approaches addressing its unique challenges. LF-Transformer (Na et al., 2024) applies transformer architectures to both columns and rows, computing an attention latent factor matrix to enhance predictions. SwitchTab (Wu et al., 2024) leverages self-supervised learning to produce improved latent representations, which lead to better downstream task performance. TabR (Gorishniy et al., 2023) introduces a neural network integrated with a custom k-Nearest-Neighbors-like component to capture feature relationships.

Unlike these models, FusANet is designed as a fully parametric model without relying on pretraining or fine-tuning techniques. Instead, FusANet directly utilizes attention mechanisms to efficiently model feature interactions and achieve superior performance.

2.2. Attention-based models for tabular data

The attention mechanism's success in Transformers (Vaswani et al., 2017) and its variants (Fnu & Bansal, 2024) have motivated its use for tabular data, resulting in the creation of various attention-based models. FT-Transformer (Gorishniy et al., 2021) treats each feature as a token and uses a transformer encoder to capture feature interactions. TabTransformer (Huang et al., 2020) focuses on categorical features, employing multiple Transformer layers to convert their embeddings into contextualized representations. TabNet (Arik & Pfister, 2019) dynamically identifies important features for each sample and employs feature masking during the pretraining phase. AutoInt (Song et al., 2018) applies self-attention to learn pairwise feature interactions. DANet (Chen et al., 2021) introduces a flexible module to group correlated features and explicitly generate higher-level feature representations. SAINT (Somepalli et al., 2021) integrates intra-sample self-attention with inter-sample attention and leverages self-supervised pretraining to enhance performance. GANet (Bui & Le, 2024) introduces global attention to further enhance the interpretability of attention-based model in tabular data.

FusANet builds on these advancements by employing a combination of attention mechanisms. Specifically, it leverages local attention, to capture feature interactions within individual samples. Additionally, FusANet uses global attention to learn shared feature relationships across the entire dataset. To integrate these complementary insights, FusANet introduces a cross-attention mechanism that fuses the outputs of local and global attention layers. This design enables the model to effectively capture both instance-specific and dataset-wide patterns. Finally, the output embeddings derived from the [CLS] token are used for prediction, enhancing both the model's performance and interpretability. A recent survey of Vision Transformer (ViT) variants catalogs the diversity of the transformer family-windowed, hierarchical, and convolutional-projection attention-developed primarily for spatial vision tasks, and helps contextualize how our global-local design differs from spatially localized attention in ViT-style models (Fnu & Bansal, 2024).

2.3. Interpretability of attention-based models

Attention mechanisms are widely recognized for their interpretability, particularly in natural language processing (NLP), where they help explain the significance of individual words in a sentence. Studies such as Chefer et al. (2021), Hao et al. (2021), Liu et al. (2022) have utilized attention weights to analyze feature importance and model decision-making and how attention can be used to obtain feature importance as a cost-effective way (Gorishniy et al., 2021).

Motivated by these findings, FusANet leverages the interpretability offered by attention layers. Unlike prior works, FusANet provides dual-layer interpretability: *Instance-Level Interpretability* - local attention weights highlight the relevance of features for individual samples. *Dataset-Level Interpretability* - global attention weights indicate the importance of each feature across the entire dataset. This dual-layer approach allows FusANet to provide actionable insights, helping users understand both instance-specific and dataset-wide feature contributions.

3. Background

3.1. Global attention layer

Given token representations $V \in \mathbb{R}^{n_v \times d_v}$, where n_v is the number of tokens and d_v is the embedding dimension, from our previous work (Bui & Le, 2024), we introduce a dataset-level, shared attention map $W_{\text{global}} \in \mathbb{R}^{n_v \times n_v}$ that can be computed by the global attention mechanism using weighted sum of V and W_{global} to capture stable affinities between tokens (features) and is reused for every instance. The attention weights are normalized using the softmax function, ensuring

interpretability:

$$\text{GlobalAtt}(V; W_{\text{global}}) = \text{softmax}(W_{\text{global}}) \cdot V.$$

Multi-Head Global Attention: Multi-head attention is utilized to improve the model's representational capacity. Each token vector V is projected into h subspaces using learnable projection matrices $W_j^V \in \mathbb{R}^{d_v \times d_v/h}$. The global attention mechanism is applied independently to each head, producing h output vectors. These outputs are concatenated and linearly transformed using $W^O \in \mathbb{R}^{(h \cdot d_v/h) \times d_v}$ to combine the information across heads:

$$\text{GlobalMultiheadAtt}(V; W_{\text{global}}) = \text{concat}[O_1, \ldots, O_h] \cdot W^O,$$

where:

$$O_j = \text{GlobalAtt}(V \cdot W_j^V; W_{\text{global}}).$$

While the shared attention map reduces redundancy by avoiding sample-specific computations, the global attention layer still has a complexity of $O(n_v^2 \cdot d_v)$ due to the matrix multiplication involving W_{global}. This complexity is consistent with other attention mechanisms, but FusANet achieves computational efficiency by pre-learning the attention map W_{global} across the dataset. We provided pseudocode of Global attention (mm is the abbreviation for matrix multiplication) in Algorithm 1.

3.2. Initialization strategy for the global attention layer

To improve the stability and interpretability of the Global Attention Layer, we enhance the initialization scheme from our prior work (Bui & Le, 2024) by adopting a hybrid approach that combines an identity matrix with Xavier initialization (Glorot & Bengio, 2010).

The shared attention map $w \in \mathbb{R}^{n_{\text{features}} \times n_{\text{features}}}$ is computed as:

$$w = \text{softmax}(w_1 + w_2), \tag{1}$$

where w_1 is an identity matrix:

$$w_1 = I_{n_{\text{features}}}, \tag{2}$$

and w_2 is a perturbation matrix initialized using Xavier initialization:

$$w_2 \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right). \tag{3}$$

This design ensures that the initial attention map starts with a feature-wise independent baseline (due to w_1), while allowing controlled learning of cross-feature dependencies through w_2. The softmax operation normalizes the resulting weights, forming the final attention distribution. We provide mathematical explanation and comparison with He initialization (He et al., 2015) in Appendix A.

Algorithm 1 Global attention block.

procedure GLOBALATTBLOCK(*input*)
  Input: $x \in \mathbb{R}^{b \times n_v \times d_v}$ ▷ Token representations
  Output: $out \in \mathbb{R}^{b \times n_v \times d_v}$
  Step 1: Normalize Input
  $x_{\text{norm}} \leftarrow \text{Norm}(x)$
  Step 2: Compute Value Projections
  $V \leftarrow \text{mm}(W_V, x_{\text{norm}})$ ▷ Shape: $b \times n_v \times d_v$
  Step 3: Apply Shared Global Attention Map
  $attn \leftarrow \text{softmax}(W_{\text{global}})$ ▷ Shared map: $W_{\text{global}} \in \mathbb{R}^{n_v \times n_v}$
  $attn_output \leftarrow \text{mm}(attn, V)$ ▷ Shape: $b \times n_v \times d_v$
  Step 4: Add Residual Connection and Feed-Forward Network
  $out_{\text{residual}} \leftarrow x + attn_output$
  $out_{\text{ffn}} \leftarrow \text{FeedForwardNetwork}(out_{\text{residual}})$
  $out \leftarrow out_{\text{residual}} + out_{\text{ffn}}$
  return *out*
end procedure

4. FusANet - Transformer for tabular data

FusANet is a novel architecture designed to model tabular data by integrating local and global attention mechanisms, fused using a cross-attention layer. At its core, FusANet utilizes two distinct [CLS] tokens (Classification token, from Devlin et al., 2019), prepended to the feature embeddings of each sample. These [CLS] tokens are used as aggregate representations for the local and global attention modules, respectively. During the attention modules' process, the [CLS] token attends to all other tokens, capturing the context for the respective modules. After processing the sample through both the global and local attention modules, FusANet fuses their outputs using a cross-attention module. Finally, predictions are constructed using the embedding of the [CLS] token from the fused representation. The following subsections provide a detailed explanation of each FusANet's components and the whole architecture.

4.1. Feature embedding block

Tabular data combines numerical and categorical features with distinct distributions, posing challenges for deep learning models to effectively represent and process these heterogeneous feature types. In FusANet, numerical and categorical features are processed separately in the feature embedding block to preserve their unique characteristics and ensure the interpretability of the model.

In detail, tabular inputs are not sequences: columns have stable semantic identities (features) but no ordering semantics. Accordingly, FusANet adapts attention as follows: No positional encodings. We do not add sinusoidal or learned positional embeddings; attention is computed without any notion of token order. Schema-aware, not order-aware. Each column corresponds to a feature with a fixed identity in the data schema. Attention weights are therefore keyed to feature identity (i.e., "blood pressure," "income," …), not to a location in a sequence.

For categorical features, FusANet employs a shared lookup table where each category is mapped to a dense, learnable embedding. This approach treats each category as a distinct entity and captures relationships across features by learning embeddings that represent their contextual meaning. In particular, all categorical columns index a single embedding matrix $E \in \mathbb{R}^{N \times d_{\text{cat}}}$.

For column f with vocabulary V_f, define a fixed offset

$$o_f = \sum_{k<f} \left(|V_k| + s_k\right),$$

where s_k counts per-field special tokens (e.g., UNK/MISS/RARE). If `sklearn` provides an in-column code $i_f(c) \in \{0, \ldots, |V_f| - 1\}$, form the global id

$$g_f(c) = o_f + i_f(c),$$

which guarantees disjoint id spaces across columns.

The embedding for column f is

$$e_f = E[g_f(c)] + e_f^{(\text{field})},$$

where $e_f^{(\text{field})}$ is an optional field (column) embedding. We then apply a shared linear projection to width d and feed the unordered set $\{e_f\}_{f=1}^{F}$ to the attention module (no positional encodings).

For numerical features, FusANet applies an element-wise matrix multiplication that transforms the raw numerical values using a learnable weight vector. This step scales and projects the numerical values into the same embedding space as the categorical features. The element-wise transformation ensures that the numerical features are effectively encoded while preserving their original values' significance.

By processing numerical and categorical features with dedicated encoders-and applying element-wise transformations to numerical features-FusANet improves predictive accuracy while *preventing cross-feature mixing prior to the attention operation*. In other words, the first weighted combination across features occurs inside the attention

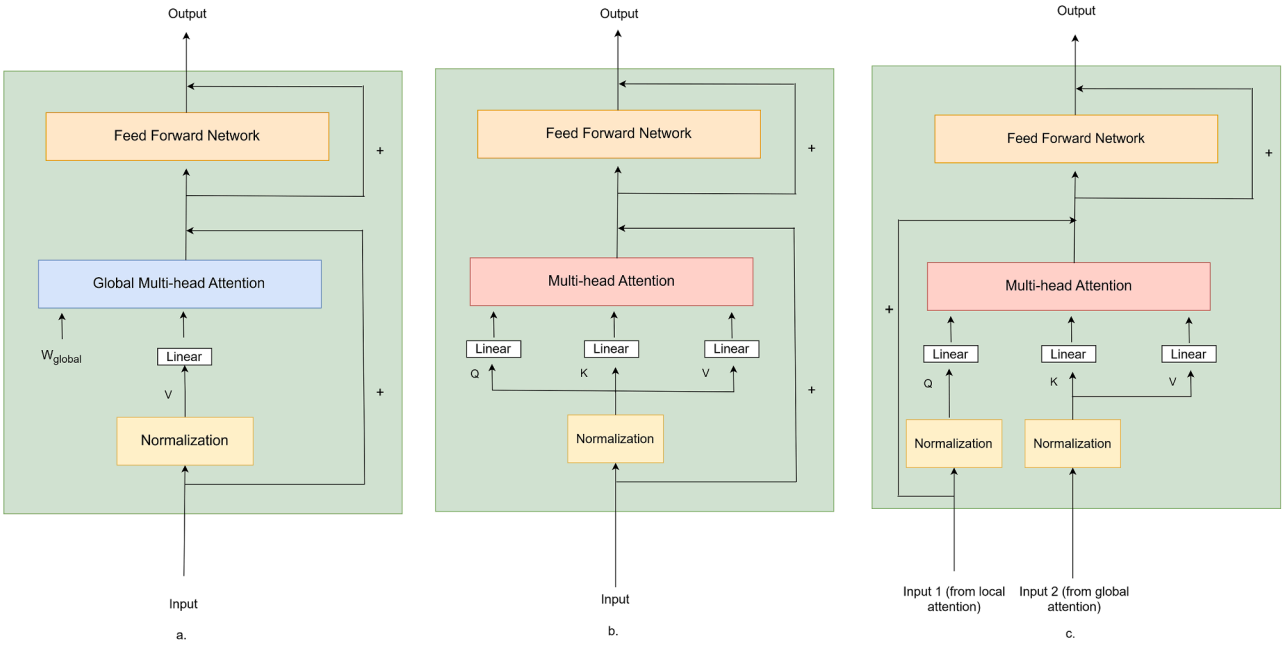


Fig. 1. The structure of (a) global attention block: : applies a shared dataset-level attention map W_{global} (softmax-normalized) to value projections; outputs preserve token order and are followed by residual + FFN. (b) local attention block: standard multi-head self-attention over feature tokens of one instance. (c) cross-attention block: queries come from the local branch and keys/values from the global branch; this fuses instance-specific and dataset-level signals.

block, which preserves clean per-feature semantics and supports interpretable analyses at both global and instance levels. This design makes FusANet effective and transparent for tabular modeling. Under this architecture, the model is also permutation-equivariant for the following reasons: (i) We tokenize each feature as a *(feature-id, value)* token and do not use positional encodings; all projection matrices are shared across tokens. In this setting, multi-head self-attention is permutation-equivariant: permuting tokens by a permutation matrix P yields $\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X)$. (ii) The prediction head pools token outputs via a symmetric aggregator (a CLS token without positions, or mean/sum pooling), which preserves prediction invariance. (iii) Consequently, the attention vector over features is equivariant: if features are permuted, the attention weights permute one-for-one, i.e., $a(PX) = P\,a(X)$.

4.2. Cross-attention layer

In standard Transformer architectures (Vaswani et al., 2017), cross-attention is used to align and transfer information between different sequences or modalities. For example, in encoder-decoder architectures for machine translation, the decoder attends to the encoder's output to determine which source tokens are most relevant for generating each target token. Similarly, in multimodal tasks such as vision-language models, cross-attention allows the model to align image regions with textual descriptions, enabling interaction across modalities. However, while our cross-attention mechanism is structurally inspired by these traditional designs, our motivation and use case are fundamentally different. Instead of aligning separate sequences or modalities, we use cross-attention within the same modality-specifically, within the feature space of tabular data. The reason behind this idea is that tabular data is inherently different from sequences or multimodal inputs: each instance consists of a fixed set of heterogeneous features, and meaningful patterns often arise from both: Instance-specific feature interactions - localized patterns unique to individual samples and global, dataset-wide correlations - statistical dependencies that remain consistent across the entire population.

To capture this dual structure, we design a cross-attention module that enables bidirectional interaction between two complementary views of the same tabular input: The *local attention representation*, which captures fine-grained, sample-specific dependencies. The *global attention representation*, which encodes shared, population-level relationships. In general, Cross-attention layer has the same structure as the usual attention layer, but instead of receiving the same input to project into 3 vectors Q_l, K_g, V_g using a linear layer as the usual self-attention layer, K_g, V_g are produced from the contextual representation of global attention module and Q_l is produced from the contextual representation of local attention module respectively (see Fig. 1(c)). In other words, FusANet uses the information of the current sample to query the shared information.

Similar to the self-attention layer, consider the representation vectors of n_v tokens $V \in \mathbb{R}^{n_v \times d_v}$. The cross-attention function CrossAtt(.) computes a weighted sum of $V \in \mathbb{R}^{n_v \times d_v}$ using the dot-product similarity between the queries Q and the corresponding keys K ($Q, K \in \mathbb{R}^{n_v \times d_q}$) for each V. This process is followed by applying the scaled softmax function $\omega(.) = \mathrm{softmax}(./\sqrt{d})$, as defined in the equation.
4:

$$\mathrm{CrossAtt}(Q, K, V; \omega) = \omega(QK^{\top})V \tag{4}$$

Specifically, by utilizing learnable parameters $W_j^Q \in \mathbb{R}^{d_q \times d_q^M}$, $W_j^K \in \mathbb{R}^{d_k \times d_k^M}$, and $W_j^V \in \mathbb{R}^{d_v \times d_v^M}$, where $d_q = d_k$ for the jth head, we apply multi-head attention to project Q_l, K_g, and V_g onto h different vectors of dimensions d_q^M, d_q^M, and d_v^M, respectively, with $d_v^M = d_v/h$ and $d_q^M = d_q/h$. The attention function CrossAtt is then applied to each of these h projections (as defined in Eq. (6)). Finally, the desired output is obtained by applying a linear transformation using $W^O \in \mathbb{R}^{h \times d_v^M \times d}$ and a feed forward network (FFN) to the concatenation of all attention outputs (as shown in Eq. (5)).

$$\mathrm{CrossMultiheadAtt}(Q_l, K_g, V_g; \omega) = \mathrm{concatenation}[O_1, \ldots, O_h]W^O \tag{5}$$

where

$$O_j = \mathrm{CrossAtt}(Q_l W_j^Q, K_g W_j^K, V_g W_j^V; W_j) \tag{6}$$

For short, cross-attention implies cross-multi-head attention. Fig. 1(c) visualizes the CrossAtt. Algorithm 2 (in which mm is abbreviation for matrix multiplication) formally shows the structure of a cross-attention

block. Similar to the local attention mechanism, Cross-attention has $O(n_v^2 d_q)$ complexity in dot similarity computation. In our experiments, we only employ 1 cross-attention block in our cross-attention module.

Algorithm 2 Cross-attention block.

procedure CROSSATTBLOCK(*input1*, *input2*)
  Input:
  $input1 \in \mathbb{R}^{b \times n_1 \times d}$ ▷ Local attention output
  $input2 \in \mathbb{R}^{b \times n_2 \times d}$ ▷ Global attention output
  Output:
  $output \in \mathbb{R}^{b \times n_1 \times d}$
  Step 1: Normalize Inputs
  $norm_input1 \leftarrow \text{Norm}(input1)$
  $norm_input2 \leftarrow \text{Norm}(input2)$
  Step 2: Compute Query, Key, and Value
  $Q \leftarrow \text{mm}(W_Q, norm_input1)$ ▷ Shape: $b \times n_1 \times d_q$
  $K \leftarrow \text{mm}(W_K, norm_input2)$ ▷ Shape: $b \times n_2 \times d_k$
  $V \leftarrow \text{mm}(W_V, norm_input2)$ ▷ Shape: $b \times n_2 \times d_v$
  Step 3: Compute Attention Weights and Output
  $attn \leftarrow \text{softmax}(\text{mm}(Q, K^\top)/\sqrt{d_k})$ ▷ Shape: $b \times n_1 \times n_2$
  $attn_output \leftarrow \text{mm}(attn, V)$ ▷ Shape: $b \times n_1 \times d_v$
  Step 4: Add Residual Connection
  $residual_connection \leftarrow input1 + attn_output$ ▷ Shape: $b \times n_1 \times d$
  Step 5: Feed-Forward Network
  $ffn_output \leftarrow \text{FeedForwardNetwork}(residual_connection)$ ▷ Shape: $b \times n_1 \times d$
  Step 6: Final Residual Connection
  $output \leftarrow residual_connection + ffn_output$ ▷ Shape: $b \times n_1 \times d$
  return *output*
end procedure

The key motivation behind this design is to improve the model's ability to simultaneously capture both local (instance-specific) and global (dataset-wide) dependencies. Traditional approaches process attention independently for each sequence, which may limit their ability to learn interactions between local and global contexts. Our approach addresses this limitation by allowing the local attention [CLS] token to attend to the global attention [CLS] token, and vice versa, enabling the model to learn more complex relationships across different levels of granularity. This integration enhances the model's ability to capture diverse patterns, improving both performance and interpretability.

4.3. Architecture

The architecture of FusANet is shown in Fig. 2. It consists of a feature embedding block, a global attention module composed of k global attention blocks, a local attention module composed of k' local attention blocks containing typical multi-head attentions from Vaswani et al. (2017), and a cross-attention layer. The final prediction is made using several fully connected linear layers. For better understand about the architecture's computational complexity, we provide deeper analysis about it in Appendix B.

To ensure stable training, FusANet employs the PreNorm variant (Wang et al., 2019) for its attention modules. The PreNorm formulation is defined in Eq. (7) and used throughout the architecture. Each attention block (global, local, and cross-attention) uses this formulation, combined with a feed forward network (FFN) defined in Eq. (8). The full attention block integrates both components as shown in Eq. (9).

$$\text{ResPreNorm}(\text{module}, x) = x + \text{Dropout}(\text{module}(\text{Norm}(x))) \tag{7}$$

$$\text{FFN}(x) = \text{Linear}(\text{Dropout}(\text{Activation}(\text{Linear}(x)))) \tag{8}$$

$$\text{Block}(x) = \text{ResPreNorm}(\text{FFN}; \text{ResPreNorm}(\text{MultiheadAtt}; x)) \tag{9}$$

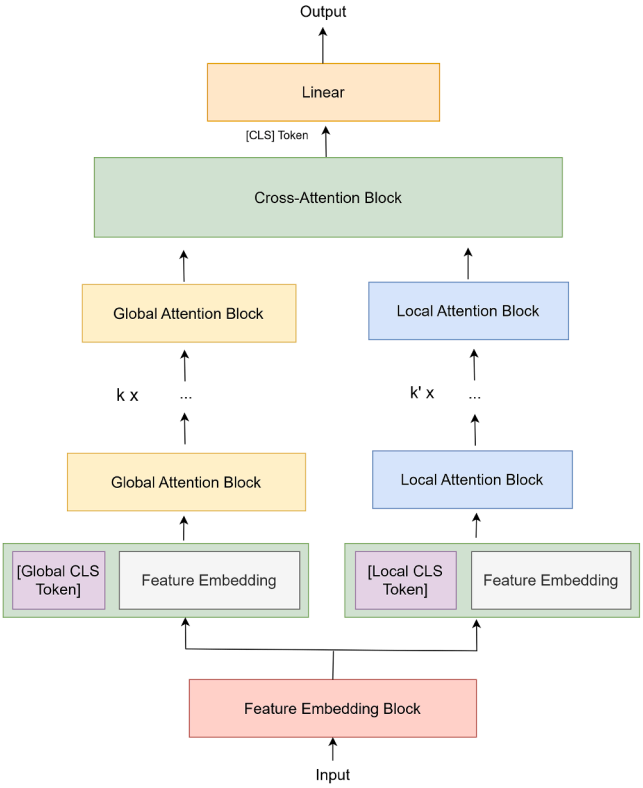


Fig. 2. FusANet architecture. Numerical and categorical features are embedded separately and concatenated with two [CLS] tokens (one per branch). A stack of k global and k' local attention blocks produce contextualized representations that are fused via one cross-attention block. The prediction head reads from the fused [CLS]. No positional encodings are used (set-based, permutation-equivariant design).

Residual pre-normalization. PreNorm ensures that normalization is applied to the input x before it is processed by the main module, stabilizing training for deep architectures: *Layer Normalization*: Normalizes the input to reduce internal covariate shift, improving gradient flow. *Dropout*: Prevents overfitting by zeroing out a fraction of the module's outputs during training. *Residual Connections*: Allow gradients to bypass the module, mitigating the risk of vanishing gradients in deep networks. PreNorm is particularly advantageous in FusANet's multi-layer attention blocks, as it ensures that all modules receive normalized inputs, improving stability compared to PostNorm variants.

Feed-forward network (FFN). The FFN applies two linear transformations with a non-linear activation function in between: The first transformation projects the input into a higher-dimensional space. The second transformation maps it back to the original dimension.

Attention Block. The full attention block combines multi-head attention and the FFN within two residual layers. The first residual layer processes the attention mechanism, and the second refines the output using the FFN. This design ensures gradient flow while enabling the model to capture both token-level dependencies and feature transformations.

Activation function. FusANet employs GeLU (Hendrycks & Gimpel, 2016) in the attention modules and ReLU (Agarap, 2018) is used for Feature Embedding Block and final linear layers.

5. Experiments

In this section, we conduct experiments on three primary tasks: binary classification, multi-class classification, and regression. The performance of FusANet is evaluated in comparison to other state-of-the-art

models, as well as to two variations of our model: G-FusANet, which omits the local attention and attention fusion modules, and L-FusANet, which omits the global attention and attention fusion modules. We also prepared an experiment about how positively fusing information by Cross-attention layer affects our model. In particular, using the aforementioned datasets, we compare the performance of FusANet when employing different information integration, which are Concatenation (denoted as Concat.), Addition (denoted as Add.), Cross-attention (denoted as Cross-att.). For Concatenation and Addition, our model employs additional linear layers followed by an activation function. After that, we demonstrate the efficiency of using our hybrid initialization using identity matrix and Xavier initialization, in comparison of that using uniform initialization from Bui and Le (2024). Finally, we outline the experimental setup and discuss the results and interpretability of FusANet in the following subsections. Additional information about the datasets and training settings can be found in Appendices C and D, respectively.

Besides the interpretability of local attention which other works have demonstrated, we demonstrate the interpretability of FusANet by the following experiment inspired by Serrano and Smith (2019): According to the sum of attention weights across heads of the global attention layer from the best performance model for each seed, we remove each feature and retrain the model using the corresponding configuration then take the average across seeds for each feature.

5.1. Benchmark

We evaluate the performance of FusANet against several state-of-the-art open-source gradient boosting algorithm and fully parametric deep learning models, including Random Forest (Breiman, 2001) as a baseline, XGBoost (Chen & Guestrin, 2016), CatBoost (Prokhorenkova et al., 2018) as strong classical methods, FT-Transformer (Gorishniy et al., 2021), DANets (Chen et al., 2022), and SAINT (Somepalli et al., 2021) as state-of-the-art deep fully-parametric models. We perform hyperparameter optimization using 10 different random seeds. The optimal hyperparameters are determined by training the model from scratch on the training subset, optimizing the loss function on both the training and validation subsets, and subsequently validating the model on the test set using the best-found hyperparameters. The final result is reported as the average performance on the test set. Hyperparameter tuning is carried out using the HyperOpt library[1]. For model evaluation, we employ two metrics: accuracy (denoted as Acc.) for classification tasks, and Root Mean Square Error (RMSE) for regression tasks.

5.2. Training and evaluation settings

We train and optimize DANet according to the procedures outlined in their respective papers. For all other deep learning models, we employ the AdamW optimizer (Loshchilov & Hutter, 2017) without a learning rate scheduler, and early stopping is applied if the validation loss does not decrease after 15 epochs. The data are split into 80 % for training and 20 % for testing, with 20 % of the training data set aside for validation. During preprocessing, we remove rows containing missing values, and categorical features are embedded to the same dimensionality, while numerical features are standardized using the Standard Scaler from the Scikit-learn library[2] for simplicity. For classification tasks, the deep neural network model is optimized using the Cross-Entropy loss function, while the Mean Squared Error loss function is used for regression tasks.

5.3. Results

Table 1 presents the benchmark results comparing different architectures, with FusANet demonstrating superior performance across various datasets. FusANet surpasses all other deep learning parametric approaches, achieving state-of-the-art results on 10 out of 10 datasets. This consistent advantage is attributed to its innovative architectural design, which effectively integrates global and local feature representations through cross-attention modules.

Table 1 delves deeper into the design choices and their effectiveness. Specifically, L-FusANet and G-FusANet excel on small datasets such as CP, BT, WC, and LD, where the number of samples is limited, and the complexity of the feature space varies significantly. On the other hand, FusANet demonstrates exceptional performance on larger and more complex datasets like AD, CA, EY, HI, CH, and GE, where the interplay of numerous numerical and categorical features requires a robust integration mechanism.

According to Table 2, the incorporation of cross-attention modules enables FusANet to achieve superior results by effectively fusing information from global and local attention mechanisms. This approach surpasses traditional integration methods like simple concatenation or the addition of [CLS] representation vectors, which fail to capture the intricate relationships across diverse feature types.

Last but not least, the Table 3 indicates that our design of initialization function for global attention allows us to achieve a better performance and more stable results compared to the original suggestion from our previous paper of Global attention.

Notably, even within the FusANet family, G-FusANet and L-FusANet deliver competitive results, frequently outperforming other state-of-the-art models. These variants highlight the flexibility of our design, adapting well to varying dataset characteristics, such as different numbers of samples, numerical and categorical feature distributions, and target metrics.

Overall, the results confirm that FusANet and its variants are not only highly effective but also versatile, adapting seamlessly to datasets of varying scales and complexities. This adaptability underscores the strength of our architectural innovations, setting a new benchmark for tabular data modeling.

5.4. Qualitative analysis

We show the global attention weights of two datasets (Adult Income Dataset in Table 4 and California Housing Dataset in Table 5) and take several samples in these datasets with corresponding local attention weights to illustrate how global and local attention interact on real data points. All global-attention and local-attention scores are outputs of a softmax over features, so each set of weights sums to 1.

5.4.1. Adult income dataset

Global attention emphasizes `education_num` (0.22) and `capital_gain` (0.18) as top predictors, implying the model relies heavily on formal education years and realized gains. However, both can act as proxies for unobserved advantages-access to elite networks or inherited wealth-rather than true causal drivers. `marital_status` (0.15) and `age` (0.10) follow, capturing broad demographic trends but risking stereotype reinforcement (e.g., “married → higher pay”). `hours_per_week` (0.10) reflects a bias toward full-time work as a salary guarantee, which overlooks high-earning part-time professionals. Surprisingly, `occupation` (0.05) is relegated low despite its intuitive importance-a sign of potential feature-encoding misalignment or overreliance on other proxies.

In Sample 1: `education_num=13` (0.18) and `capital_gain=5178` (0.12) align with these global priors, but local attention sharply up-weights `occupation = Exec-managerial` (0.20) and `hours_per_week=40` (0.15). This tacitly assumes managerial titles and full-time roles always equate to high pay, ignoring well-paid non-executives or part-time specialists. Meanwhile, `marital_status = Married-civ-spouse` (0.08) and `age=42` (0.07) receive less emphasis than one might expect given demographic income studies-highlighting inconsistent weighting versus known trends.

[1] https://github.com/hyperopt/hyperopt
[2] https://scikit-learn.org/stable/

Table 1
vBenchmarks on 10 tabular datasets (10 seeds; mean reported). Arrows in the header indicate the desirable direction (↑ higher is better, ↓ lower is better). Bold = best overall; underline = best among deep parametric models. Classical baselines (RF/XGBoost/CatBoost) are included for context; FusANet variants (G-/L-) test the contribution of global/local branches and fusion. The "Rank" column reports the mean rank across datasets.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑	Rank
Random Forest	2.486	0.775	0.942	**0.440**	0.858	0.536	0.637	0.705	0.861	0.605	6.9
XGBoost	**2.107**	0.753	0.947	0.462	**0.873**	**0.438**	**0.735**	0.728	0.860	**0.688**	3.9
CatBoost	2.133	0.780	0.950	0.447	0.871	0.442	0.690	0.726	0.855	0.685	4.4
DANet	2.711	0.793	0.953	0.470	0.846	0.579	0.574	0.723	0.859	0.579	7.2
SAINT	2.245	0.784	0.956	0.470	0.860	0.481	0.650	0.730	0.858	0.612	5.5
FTTransformer	2.175	0.785	0.949	0.474	0.859	0.476	0.699	0.732	0.861	0.623	4.9
G-FusANet	2.286	0.792	**0.961**	0.442	0.861	0.470	0.660	0.730	0.861	0.601	4.1
L-FusANet	2.194	**0.795**	0.952	0.470	0.859	0.475	0.709	0.730	0.862	0.625	4.0
FusANet	2.192	0.779	0.954	0.459	0.865	0.466	0.715	**0.735**	**0.865**	0.636	3.1

Table 2
Effect of information integration. We compare Addition and Concatenation of [CLS] representations against Cross-attention fusion. All settings share the same search space and training protocol as Table 1; Cross-attention yields the strongest average results, indicating that content-aware fusion outperforms static merges.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
Add.	2.222	**0.795**	0.951	0.470	0.861	0.473	0.703	0.732	0.863	0.620
Concat.	2.213	**0.795**	0.949	0.466	0.859	0.471	0.700	0.733	0.863	0.615
CrossAtt.	**2.192**	0.779	**0.954**	**0.459**	**0.865**	**0.466**	**0.715**	**0.735**	**0.865**	**0.636**

Table 3
Initialization of the global attention map. Uniform = the uniform scheme from our prior global-attention work; Hybrid = identity + Xavier. Hybrid produces consistently better or equal performance and lower variance across seeds.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
Uniform	2.220	0.772	0.948	0.465	0.864	0.471	0.711	0.734	0.860	0.630
Hybrid	**2.192**	**0.779**	**0.954**	**0.459**	**0.865**	**0.466**	**0.715**	**0.735**	**0.865**	**0.636**

Table 4
Adult Income - Global and Local Attention (softmax). The Global Weight column reports dataset-level feature importance; each local column shows per-sample attention with the raw feature value or category in parentheses. We predict whether `income` is >50K or ≤50K. Weights are softmax-normalized and sum to 1 per column. Samples 1–3 illustrate contrasting outcomes and how local attention departs from global ranks.

Feature	Global Weight	Sample 1 income = >50K	Sample 2 income = ≤50K	Sample 3 income = >50K
education_num	0.22	0.18 (13)	0.12 (4)	0.15 (10)
capital_gain	0.18	0.12 (5178)	0.05 (0)	0.08 (0)
marital_status	0.15	0.08 (Married-civ-spouse)	0.20 (Married-civ-spouse)	0.10 (Married-civ-spouse)
age	0.10	0.07 (42)	0.09 (34)	0.06 (37)
hours_per_week	0.10	0.15 (40)	0.18 (45)	0.27 (80)
workclass	0.05	0.05 (Private)	0.04 (Private)	0.03 (Private)
occupation	0.05	0.20 (Exec-managerial)	0.15 (Transport-moving)	0.20 (Exec-managerial)
relationship	0.04	0.05 (Husband)	0.06 (Husband)	0.04 (Husband)
education	0.03	0.03 (Bachelors)	0.04 (7th-8th)	0.02 (Some-college)
capital_loss	0.03	0.01 (0)	0.02 (0)	0.01 (0)
native_country	0.02	0.03 (United-States)	0.02 (Mexico)	0.01 (United-States)
fnlwgt	0.01	0.01 (159449)	0.01 (245487)	0.01 (280464)
race	0.01	0.01 (White)	0.01 (Amer-Indian-Eskimo)	0.01 (Black)
sex	0.01	0.01 (Male)	0.01 (Male)	0.01 (Male)

In Sample 2: `education_num=4` (0.12) and `capital_gain=0` (0.05) drive a "≤50K" prediction, reflecting a blanket bias that minimal education equates low income. Yet many skilled tradespeople earn above 50K with only vocational training-the model misses such nuance. Local attention increases `marital_status=Married-civ-spouse` (0.20) and `hours_per_week=45` (0.18), but they cannot overcome the global bias against low education, showing that local tweaks struggle against rigid priors.

In Sample 3: `education_num=10` (0.15) and `capital_gain=0` (0.08) become secondary to extreme `hours_per_week=80` (0.27) and `occupation = Exec-managerial` (0.20). The model effectively says "80 hours + exec role → high income," which can hold statistically but fails for overworked mid-level managers in startups who still earn below 50K. `marital_status = Married-civ-spouse` (0.10) and `age=37` (0.06) have minimal impact, illustrating brittle assumptions about workforce dynamics.

5.4.2. *California housing dataset*

Global attention emphasizes `median_income` (0.25), `households` (0.15), and `ocean_proximity` (0.15) as key drivers of home value. Yet, this ranking may hide confounding factors like gentrification or zon-

Table 5
California Housing - Global and Local Attention (softmax). The Global Weight column gives dataset-level importance; each local column shows per-block attention with the observed feature value in parentheses. The target `value` is median house value (USD). Weights are softmax-normalized and sum to 1 per column. Samples A-C illustrate inland vs. coastal settings and varying income levels that shift local attention relative to global ranks.

Feature	Global Weight	Sample A value = 261100	Sample B value = 158400	Sample C value = 500001
median_income	0.25	0.10 (3.6912)	0.18 (3.5532)	0.20 (13.466)
ocean_proximity	0.15	0.20 (NEAR BAY)	0.10 (INLAND)	0.18 (<1H OCEAN)
households	0.15	0.18 (714)	0.15 (372)	0.10 (326)
longitude	0.12	0.12 (-122.25)	0.15 (-121.77)	0.12 (-117.79)
latitude	0.10	0.10 (37.84)	0.10 (37.68)	0.10 (33.75)
total_rooms	0.08	0.08 (3549)	0.08 (1687)	0.08 (2893)
population	0.06	0.15 (1551)	0.12 (950)	0.12 (983)
total_bedrooms	0.05	0.05 (707)	0.06 (372)	0.06 (345)
housing_median_age	0.04	0.02 (52.0)	0.06 (36.0)	0.04 (26.0)

ing laws. Local attention attempts to re-weight these according to each block's specifics, but some locality nuances remain uncaptured.

In Sample A: `median_income=3.6912` (0.10) seems low for a $261,100 home, but `ocean_proximity=NEAR BAY` (0.20) and `households=714` (0.18) drive the prediction. The model assumes bay-adjacent scarcity justifies price, ignoring rapid local developments. `population=1551` (0.15) reinforces scarcity, yet static census data can't capture short-term demographic shifts. Coordinates (`longitude=-122.25`, 0.12; `latitude=37.84`, 0.10) align with Bay Area norms but may fail if urban sprawl expands inland.

In Sample B: `median_income=3.5532` (0.18) is up-weighted over its global 0.25, indicating that here, income better explains the $158,400 valuation. `ocean_proximity=INLAND` (0.10) and `longitude=-121.77` (0.15) reflect a Central California suburb where geography trumps coast. However, excluding factors like job growth or school quality risks overfitting noise-these local weights lack economic context.

In Sample C: `median_income=13.466` (0.20) and `ocean_proximity=<1H OCEAN` (0.18) nearly tie, implying the model views wealth and location almost equally for a $500,001 home. But static features-`population=983` (0.12), `households=326` (0.10), `total_rooms=2893` (0.08), `total_bedrooms=345` (0.06), `housing_median_age=26.0` (0.04)-remain low, missing nuances like recent renovations or flood risks. Coastal premium may be over-emphasized given environmental threats.

Summary. Local attention meaningfully adjusts global importance based on contextual feature values, but it does not eliminate biases or account for dynamic shifts. In the Adult dataset, the model's heavy emphasis on occupation and hours can overlook outliers or changing labor markets. Very low education still predisposes toward "≤50K," ignoring alternative income paths. In the Housing dataset, geographic scarcity (ocean proximity) can overshadow even extreme income levels, yet it fails to anticipate future urban development or environmental risks. Transformer-based models offer contextualized feature relevance, but they remain anchored to historical data and global priors. Future work should incorporate temporal dynamics, causal inference, and external economic indicators to avoid bias in global attention weights.

5.5. Interpretability

Since FusANet preprocesses each feature separately, we can derive its interpretability from the first attention layer. By inputting a sample and taking the output of the softmax function, we obtain weights that indicate the importance of each feature for that input. Specifically, we could use the [CLS] token to explain the importance of a specific feature by considering the softmax result of the first self-attention from the local attention module as the relevancy map, which is efficient in certain cases (Hao et al., 2021). Additionally, our model offers another perspective based on the global attention layers, which can be directly extracted by taking the softmax from the first global attention layer's weights.

As shown in Fig. 3, features with higher global weights are more important than those with lower attention weights. This is demonstrated by the significant drop in FusANet's performance when these features are removed-resulting in decreased accuracy for classification tasks and increased loss for regression tasks, from left to right. The multi-head attention mechanism in both global and local attention layers enables FusANet to attend to different features in varied ways, effectively highlighting the relationships between groups of features at both global and local levels.

In addition to directly using the softmax output (attention weights) as feature importance scores, FusANet can be applied to various explanation methods for understanding tabular data, such as those using attention weights, gradients, and more (see Chefer et al., 2021, Hao et al., 2021) for both local attention layer and global attention layer.

5.6. Limitations

For high-dimensional tables, computation scales with the number of features d; if pairwise interactions are modeled, the worst-case cost approaches $\mathcal{O}(d^2)$. To improve deployability under resource or latency constraints, several *compatible* strategies preserve the task while reducing cost: (i) structured pruning of FFN/projection layers with brief fine-tuning (Sanh et al., 2020); (ii) post-training or quantization-aware training of linear layers to INT8 (Jacob et al., 2018); (iii) low-rank factorization and linear/low-rank attention for pairwise modules (e.g., Linformer/Performer) (Choromanski et al., 2020; Wang et al., 2020); (iv) learned feature gating for input pruning (e.g., L0/Concrete) (Louizos et al., 2017); and (v) knowledge distillation to a compact student while retaining the teacher's global attention as a lightweight diagnostic head (Hinton et al., 2015). In practice, the *global* attention in FusANet is dataset-shared and can be precomputed and cached with periodic refresh, further lowering online latency.

Finally, one must remain cautious: softmax outputs can obscure whether a feature is truly causal or simply correlated when we naively apply it in certain cases (Serrano & Smith, 2019) (Gorishniy et al., 2021). We therefore recommend reading attention via top-k ranks, separate local from global signals, and pair weights with simple statistics. As future work, we outline lightweight faithfulness checks (stability, triangulation with SHAP (Lundberg & Lee, 2017), and sanity tests) and explore causal-aware regularization to improve faithfulness without changing the task (Alvarez Melis & Jaakkola, 2018; Rohekar et al., 2023).

5.7. Applicability

FusANet was designed as a general, fully-parametric model for supervised tabular learning, and it applies to both classification and regression

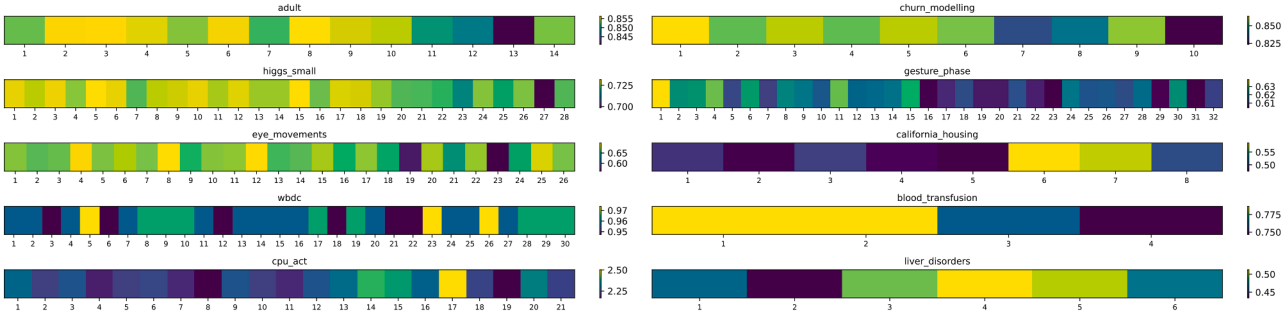


Fig. 3. Performance of FusANet after removing each feature. The x-axis indicates features whose attention weights are sorted ascendingly according to attention from left to right. California Housing, CPU Activity and Liver Disorders dataset heatmap shows the loss, the remains show the accuracy.

tasks. The model is most suitable when: The dataset contains a mix of numerical and categorical features and the practitioner wants to capture complex, possibly higher-order feature interactions that standard linear or tree-based methods may miss. Instance-level (local) and dataset-level (global) interpretability are helpful for diagnostics, fairness checks, or human-in-the-loop decision making; FusANet's dual-attention design yields relative feature relevance at both levels. Data sizes are medium to large (from a few thousand to millions of rows). In very small-data regimes, simplified variants (e.g., G-FusANet / L-FusANet) or classical methods may be more robust; we include experiments showing how FusANet compares across dataset scales (Section 5.3).

FusANet is less appropriate when: Very low-latency or extreme resource constraints are the primary requirement (edge devices, strict real-time constraints) unless compression (pruning, distillation, quantization) is applied. Extremely high-dimensional feature spaces (tens of thousands of features) where the quadratic cost of attention over features becomes prohibitive. In such settings, practitioners should first apply feature selection or use sparse/approximate attention variants.

6. Conclusion

We presented FusANet, a fully parametric, transformer-style model for tabular data that fuses a shared dataset-level attention with instance-level self-attention via a single cross-attention layer. Across 10 datasets, FusANet surpasses strong fully-parametric state-of-the-art models and is competitive with boosting, while offering dual-level attributions (dataset-wide and instance-specific). The ablations indicate that fusion-not merely adding more blocks-accounts for much of the gain and our way of initialization for global attention also improve the performance. Importantly, we showed FusANet is best suited to medium-scale and non-strict-latency deployments; when the number of features is very large or latency is critical, we could use pragmatic compression (pruning/quantization/distillation) and sparse/low-rank attention variants. Looking ahead, we will pair attention with faithfulness checks and counterfactual probes, and explore causal-aware regularization, so that attention weights remain diagnostic and more faithful without changing the supervised task.

CRediT authorship contribution statement

Hung Bui: Conceptualization, Methodology, Software, Formal analysis, Visualization, Writing – Original draft and Editing.

Bac Le: Conceptualization, Methodology, Supervision, Validation, Review & Editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by Vietnam's National Foundation for Science and Technology Development (NAFOSTED) under Grant Number 102.05-2023.44.

Appendix A. Mathematical explanation for global attention layer initialization

Mathematical explanation In the global attention layer, we construct a shared attention map $w \in \mathbb{R}^{n_{\text{features}} \times n_{\text{features}}}$ that controls how each feature attends to others across all input instances. To initialize w, we define:

$$w = \text{softmax}(w_1 + w_2),$$

where $w_1 = I_{n_{\text{features}}}$ and $w_2 \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$ follows Xavier initialization.

This initialization scheme creates a weight matrix that: 1. Starts close to an identity matrix - w_1 ensures that features initially only attend to themselves. 2. Includes small random perturbations - w_2 introduces symmetric, zero-centered noise that enables the model to learn cross-feature interactions.

Variance Analysis. For an input vector $x \in \mathbb{R}^{n_{\text{features}}}$ with zero mean and variance $\text{Var}(x_i) = \sigma^2$, the linear transformation prior to softmax becomes:

$$y = (I + \Delta W)x,$$

with $\Delta W = w_2$. Assuming independence between ΔW and x, the output variance is:

$$\text{Var}(y_i) \approx \text{Var}(x_i) + \text{Var}(\Delta W x)_i \approx \sigma^2 + \frac{2\sigma^2}{n_{\text{in}} + n_{\text{out}}}.$$

This preserves the scale of the input signal while allowing a small controlled deviation for learning.

Gradient Stability. The Jacobian of this operation with respect to x is:

$$\frac{\partial y}{\partial x} = I + \Delta W.$$

Since ΔW is small and zero-centered, the gradient flow is stable - gradients neither vanish nor explode, even across stacked attention layers. which is particularly important in our case, where poor initialization can lead to highly entangled gradient paths in attention-based networks.

Comparison with He Initialization (He et al., 2015). He initialization is a method to initialize weights in the neural network, which is

designed for use with ReLU (or similar) nonlinear activations that zero out half the input values. It compensates for this by setting a higher variance:

$$\mathrm{Var}(W_{ij}) = \frac{2}{n_{\text{in}}}.$$

However, this assumption does not hold in attention layers, especially when the attention weights are later normalized by softmax. Using He initialization in our context can introduce excessive variance, leading to: sharp or unstable softmax distributions early in training and poor signal preservation, as off-diagonal entries can dominate identity structure. By contrast, Xavier initialization assumes symmetric activation functions (like tanh or identity), and is more appropriate when both forward and backward variances should be preserved:

$$\mathrm{Var}(W_{ij}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

This is especially suitable for attention maps, where: 1. All input and output nodes are equally important (features attending to features). 2. Initial mapping should be close to identity (for interpretability). 3. Controlled randomness should be symmetric and variance-balanced. We demonstrated in Section (E.3) that our hybrid initialization strategy leads to measurable gains in accuracy and training stability across multiple datasets. These results validate our theoretical reasoning and demonstrate the practical benefits of using identity + Xavier over more generic initialization schemes.

Appendix B. Computational complexity

Setup. Let F be the number of feature tokens, D the *model width* (token embedding/hidden dimension), h the number of heads, and r the FFN expansion ratio. We count matrix-multiply FLOPs with the convention *one multiply-add = 1 FLOP*; a matrix product $A \in \mathbb{R}^{M\times K}$, $B \in \mathbb{R}^{K\times N}$ costs MKN FLOPs. We report costs in terms of the total width D; per-head width D/h cancels in sums for multi-head blocks. We count matmul FLOPs only and ignore LayerNorm/Dropout. Unless stated otherwise, attention operates on all F tokens. All attention blocks are *multi-head*.

Self-attention (full tokens, per layer).

$$C_{\text{self}} = \underbrace{2F^2D}_{\text{scores\&valuemixing}} + \underbrace{(4+2r)FD^2}_{\text{Q,K,V,}W_O\text{,FFN}}.$$

Global attention with shared W_{global} (multi-head, per layer).. This block uses a shared global map applied to values (no per-layer Q, K projections). It includes a value projection, an output projection W_O, and an FFN:

$$C_{\text{global}} = \underbrace{F^2D}_{W_{\text{global}}V} + \underbrace{(2+2r)FD^2}_{V\text{-proj.},W_O,\text{FFN}}.$$

B.0.0.1. Cross-attention (multi-head, per layer). Let n_1 be the query length and n_2 the key/value length.

$$C_\times = \underbrace{2n_1n_2D}_{\text{scores\&valuemixing}} + \underbrace{(2n_1+2n_2)D^2}_{\text{Q on }n_1,\ K,V\text{ on }n_2,\ W_O\text{ on }n_1} + \underbrace{2rn_1D^2}_{\text{FFN on queries}}.$$

Special cases: (i) CLS-to-CLS ($n_1 = n_2 = 1$): $C_\times = 2D + (4+2r)D^2$. (ii) Full sequences ($n_1 = n_2 = F$): $C_\times = 2F^2D + (4+2r)FD^2$.

Total (per instance). For L_{self} self-attention layers, L_{global} global layers, and $L_\times$ cross-attention layers,

$$C_{\text{total}} = L_{\text{self}}C_{\text{self}} + L_{\text{global}}C_{\text{global}} + L_\times C_\times.$$

Memory note. Attention maps require $O(hF^2)$ memory for self/global attention and $O(hn_1n_2)$ for cross-attention.

Conclusion. The dominant terms are quadratic in the token count F (the F^2D parts from attention score computation and value mixing) and linear in F but quadratic in width D (the FD^2 parts from projections and FFN). When F is large, F^2D dominates; when D or the FFN ratio r is large, $(4+2r)FD^2$ becomes the bottleneck. Global attention with shared W_{global} removes per-layer F^2 *parameters* but not the F^2D *compute*; in the multi-head configuration, the output projection W_O is included, contributing an additional FD^2 term. Cross-attention can be made cheap by restricting to CLS-to-CLS, which eliminates F^2 scaling. For compute- or memory-limited deployments, prefer (i) smaller F via feature grouping or learned pooling, (ii) moderate D and r, and (iii) CLS-only or sparse cross-attention, which together keep FLOPs ($\propto F^2D + FD^2$) and attention-map memory ($\propto hF^2$ or hn_1n_2) within budget.

Appendix C. Dataset

We evaluate FusANet on 10 datasets: 4 for binary classification tasks, 3 for multi-class classification tasks, and 3 for regression tasks. The datasets include Adult (Kohavi, 1996) (AD, income prediction), Blood Transfusion (Yeh et al., 2009) (BT, Blood Transfusion Service Center Data Set), Breast Cancer[3] (WC, Breast Cancer Wisconsin Data Set), California Housing (Kelley Pace & Barry, 1997) (CA, real estate data), CPU Activity[4] (a collection of computer systems activity measures), Churn Modeling [5] (CH, detailed customer information from a bank indicating whether the customer left), Eye Movements (Salojärvi et al., 2005) (EY, inferring relevance from eye movements), Gesture Phase Prediction (Madeo et al., 2013) (GE, classifying gesture phases), Higgs Small (Baldi et al., 2014) (HI, simulated physical particles, using a subset of 98,050 instances from [6], Liver Disorders (McDermott & Forsyth, 2016) (LD, BUPA liver disorders), and finally, the Adult dataset (Kohavi, 1996) (AD, income prediction), as summarized in Table C.1.

[3] https://pages.cs.wisc.edu/~olvi/uwmp/cancer.{html}
[4] http://www.ncc.up.pt/~ltorgo/Regression/DataSets.{html}
[5] https://www.kaggle.com/datasets/shrutimechlearn/churn-modeling
[6] https://www.openml.org/

Table C.1
Statistics of benchmarking datasets.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
#samples	8192	748	569	345	48842	20640	10936	98,049	10,000	9873
#numerical features	21	4	30	6	6	8	26	29	5	32
#categorical features	0	0	0	0	8	0	0	0	5	1
metric	rmse	acc.	acc.	rmse	acc.	rmse	acc	acc	acc	acc
#classes	–	2	2	–	2	–	3	2	2	5

Appendix D. Training settings

We use multiple ways to determine the range of hyperparameter searching space. In below tables, we denotes:

- IntqUniform[a;b;c]: Quantized uniform distribution from a to b with step size c.
- Uniform[a,b]: Continuous uniform distribution from a to b.
- LogUniform[a,b]: Log-uniform distribution over the interval $[e^a; e^b]$
- Choice[a,…,b]: Choose an element in a list [a,…,b] in a uniform distribution for each element

The hyperparameters space used for searching the best hyperparameters for all models is provided below:

FusANet In Table D.1, we provide the hyperparameter space used for searching the best hyperparameters for FusANet. We fixed the number of heads at 8 and the batch size at 2048. For simplicity in finding optimal hyperparameters, we use the same dropout rate of the Feed forward Network as that of previous attention layers. Additionally, in experiments related to information fusion comparison, they shared the same hyperparameter searching space, except that Concatenation and Addition did not have that for the cross-attention layer.

G-FusANet In Table D.2, we outline the hyperparameter space used to find the optimal settings for G-FusANet. The number of heads is set to 8, and the batch size is fixed at 2048.

L-FusANet In Table D.3, we outline the hyperparameter space used to find the optimal settings for L-FusANet. The number of heads is set to 8, and the batch size is fixed at 2048.

Table D.1
Search hyperparameter space for FusANet.

Hyperparameters	Distribution
# local attention Layers	IntqUniform[1; 4; 1]
# global attention Layers	IntqUniform[1; 4; 1]
Embedding dimension	IntqUniform[16; 256; 8]
Feed forward factor	Uniform[1/3; 6/3]
local attention dropout	Uniform[0; 0.5]
global attention dropout	Uniform[0; 0.5]
cross-attention dropout	Uniform[0; 0.5]
Learning rate	LogUniform[$1e-5$; $1e-3$]
Weight decay	LogUniform[$1e-6$; $1e-3$]
#Searching Iteration	50

Table D.2
Search hyperparameter space for G-FusANet.

Hyperparameters	Distribution
# global attention Layers	IntqUniform[1; 8; 1]
Embedding dimension	IntqUniform[16; 256; 4]
Feed forward factor	Uniform[1/3; 8/3]
attention dropout	Uniform[0; 0.5]
Feed forward dropout	Uniform[0; 0.5]
Learning rate	LogUniform[$1e-5$; $1e-3$]
Weight decay	LogUniform[$1e-6$; $1e-3$]
#Searching Iteration	50

Table D.3
Search hyperparameter space for L-FusANet.

Hyperparameters	Distribution
# local attention Layers	IntqUniform[1; 8; 1]
Embedding dimension	IntqUniform[16; 256; 4]
Feed forward factor	Uniform[1/3; 8/3]
attention dropout	Uniform[0; 0.5]
Feed forward dropout	Uniform[0; 0.5]
Learning rate	LogUniform[$1e-5$; $1e-3$]
Weight decay	LogUniform[$1e-6$; $1e-3$]
#Searching Iteration	50

Table D.4
Search hyperparameter space for FT Transformer.

Hyperparameters	Distribution
#Layers	IntqUniform[1; 4; 1]
Embedding dimension	IntqUniform[32; 256; 8]
Feed forward factor	Uniform[2/3; 8/3]
attention dropout	Uniform[0; 0.5]
Feed forward dropout	Uniform[0; 0.5]
Learning rate	LogUniform[$1e-5$; $1e-3$]
Weight decay	LogUniform[$1e-6$; $1e-3$]
#Searching Iteration	50

Table D.5
Search hyperparameter space for DANet.

Hyperparameters	Distribution
n_d	IntqUniform[8; 64; 8]
n_steps	Uniform[2/3; 8/3]
gamma	Uniform[0; 0.5]
n_independent	Uniform[0; 0.5]
n_shared	Uniform[0; 0.5]
Learning rate	LogUniform[$1e-5$; $1e-3$]
#Searching Iteration	50

FT Transformer In Table D.4, we present the hyperparameter space utilized when searching for the optimal hyperparameters for FT Transformer. The number of heads is fixed at 8, and the batch size is set to 2048.

DANet In Table D.5, we outline the hyperparameter space utilized during the search for the optimal hyperparameters for DANet. The virtual batch size is fixed at 256, and the batch size is set to 2048.

XGBoost In Table D.6, we outline the hyperparameter space used during the search for the optimal hyperparameters for XGBoost. The number of estimators is fixed at 2000, early_stopping_rounds to 50, tree_method to “gpu_hist”

CatBoost In Table D.7, we outline the hyperparameter space used during the search for the optimal hyperparameters for Catboost. The number of estimators is fixed at 2000, early_stopping_rounds to 50, tree_method to “gpu_hist”

Random Forest In Table D.8, we outline the hyperparameter space used during the search for the optimal hyperparameters for Random Forest. The number of estimators is fixed at 2000, early_stopping_rounds to 50, tree_method to “gpu_hist”

Table D.6
Search hyperparameter space for XGBoost.

Hyperparameters	Distribution
max_depth	IntqUniform[3; 10; 1]
learning_rate	LogUniform[$1e-5$; 1]
gamma	LogUniform[$1e-8$; $1e2$]
reg_lambda	LogUniform[$1e-8$; $1e2$]
reg_alpha	LogUniform[$1e-8$; $1e2$]
min_child_weight	LogUniform[$1e-8$; $1e2$]
colsample_bytree	Uniform[0.5; 1]
colsample_bylevel	Uniform[0.5; 1]
#Searching Iteration	50

Table D.7
Search hyperparameter space for Catboost.

Hyperparameters	Distribution
max_depth	IntqUniform[3; 12; 1]
learning_rate	LogUniform[$1e-5$; $3e-1$]
l2_leaf_reg	LogUniform[$1e-2$; $1e3$]
bagging_temperature	Uniform[0; 10]
rsm	Uniform[0.5; 1]
min_data_in_leaf	LogUniform[1; 512]
random_strength	LogUniform[0; 3]
border_count	IntqUniform[32; 255; 1]
#Searching Iteration	50

Table D.8
Search hyperparameter space for Random Forest.

Hyperparameters	Distribution
max_depth	IntqUniform[2; 10; 1]
min_samples_split	IntqUniform[2; 20; 1]
min_samples_leaf	IntqUniform[2; 10; 1]
max_features	Choice[sqrt; log2, None]
bootstrap	Choice[True; False]
#Searching Iteration	50

Table D.9
Search hyperparameter space for SAINT.

Hyperparameters	Distribution
#Layers	IntqUniform[1; 4; 1]
Dimension per head	IntqUniform[32; 256; 8]
Feed forward factor	Uniform[2/3; 8/3]
attention dropout	Uniform[0; 0.5]
Feed forward dropout	Uniform[0; 0.5]
Learning rate	LogUniform[$3e-5$; $1e-3$]
Weight decay	LogUniform[$1e-6$; $1e-4$]
#Searching Iteration	50

SAINT In Table D.9, we present the hyperparameter space utilized during the search for the optimal hyperparameters for SAINT. We employ row-column attention, and the number of heads is fixed at 8. Additionally, the batch size is set to 2048.

Appendix E. Detailed performance

We show the detailed table with average results with corresponding standard deviation between FusANet and other state-of-the-arts in Table E.1, detailed benchmark results between different methods of integrating information in Table E.2 and detailed benchmark results between different methods of initialization for Global Attention in Table E.3:

Table E.1
Average result and standard deviation (subscripted) from training and averaging the results across 10 different seeds, for FusANet and other state-of-the-art parametric models. We highlight the best result in bold and the best result among deep learning models in underline.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
Random Forest	$2.486_{\pm 0.0654}$	$0.775_{\pm 0.0251}$	$0.942_{\pm 0.0212}$	**$0.440_{\pm 0.0221}$**	$0.858_{\pm 0.0040}$	$0.536_{\pm 0.0031}$	$0.637_{\pm 0.0140}$	$0.705_{\pm 0.0011}$	$0.861_{\pm 0.0037}$	$0.605_{\pm 0.0083}$
XGBoost	**$2.107_{\pm 0.0256}$**	$0.753_{\pm 0.0262}$	$0.947_{\pm 0.0147}$	$0.462_{\pm 0.0202}$	**$0.873_{\pm 0.0025}$**	**$0.438_{\pm 0.0023}$**	**$0.735_{\pm 0.0057}$**	$0.728_{\pm 0.0028}$	$0.860_{\pm 0.0034}$	**$0.688_{\pm 0.0175}$**
CatBoost	$2.133_{\pm 0.0315}$	$0.780_{\pm 0.0328}$	$0.950_{\pm 0.0181}$	$0.447_{\pm 0.0327}$	$0.871_{\pm 0.004}$	$0.442_{\pm 0.0017}$	$0.690_{\pm 0.0420}$	$0.726_{\pm 0.0025}$	$0.855_{\pm 0.0039}$	$0.685_{\pm 0.0134}$
DANet	$2.711_{\pm 0.1853}$	$0.793_{\pm 0.0352}$	$0.953_{\pm 0.0119}$	$0.470_{\pm 0.0711}$	$0.846_{\pm 0.0032}$	$0.579_{\pm 0.0041}$	$0.574_{\pm 0.0113}$	$0.723_{\pm 0.0030}$	$0.859_{\pm 0.0021}$	$0.579_{\pm 0.0060}$
SAINT	$2.245_{\pm 0.0963}$	$0.784_{\pm 0.0191}$	$0.956_{\pm 0.0235}$	$0.470_{\pm 0.0307}$	$0.860_{\pm 0.0024}$	$0.481_{\pm 0.0060}$	$0.650_{\pm 0.0101}$	$0.730_{\pm 0.0051}$	$0.858_{\pm 0.0023}$	$0.612_{\pm 0.0045}$
FTTransformer	$\underline{2.175 \pm 0.0657}$	$0.785_{\pm 0.0375}$	$0.949_{\pm 0.0187}$	$0.474_{\pm 0.0320}$	$0.859_{\pm 0.0019}$	$0.476_{\pm 0.0020}$	$0.699_{\pm 0.0045}$	$0.732_{\pm 0.0080}$	$0.861_{\pm 0.0033}$	$0.623_{\pm 0.0222}$
G-FusANet	$2.286_{\pm 0.0581}$	$0.792_{\pm 0.0176}$	$\underline{0.961 \pm 0.0119}$	$\underline{0.442 \pm 0.0246}$	$0.861_{\pm 0.0019}$	$0.470_{\pm 0.0023}$	$0.660_{\pm 0.0013}$	$0.730_{\pm 0.0045}$	$0.861_{\pm 0.0011}$	$0.601_{\pm 0.0139}$
L-FusANet	$2.194_{\pm 0.0704}$	$\underline{0.795 \pm 0.0296}$	$0.952_{\pm 0.0133}$	$0.470_{\pm 0.0384}$	$0.859_{\pm 0.0046}$	$0.475_{\pm 0.0065}$	$0.709_{\pm 0.0060}$	$0.730_{\pm 0.0008}$	$0.862_{\pm 0.0025}$	$0.625_{\pm 0.0141}$
FusANet	$2.192_{\pm 0.0614}$	$0.779_{\pm 0.0158}$	$0.954_{\pm 0.0197}$	$0.459_{\pm 0.0354}$	$\underline{0.865 \pm 0.0030}$	$\underline{0.466 \pm 0.0041}$	$\underline{0.715 \pm 0.0096}$	$\underline{\mathbf{0.735} \pm 0.0053}$	$\underline{\mathbf{0.865} \pm 0.0032}$	$\underline{0.636 \pm 0.0092}$

Table E.2
Average results and standard deviation (subscripted) over 10 different seeds, for FusANet with different methods of information integration. We highlight the best result in bold.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
FusANet	**2.192**$_{\pm 0.0614}$	0.779$_{\pm 0.0158}$	**0.954**$_{\pm 0.0191}$	**0.459**$_{\pm 0.0354}$	**0.865**$_{\pm 0.0030}$	**0.466**$_{\pm 0.0041}$	**0.715**$_{\pm 0.0096}$	**0.735**$_{\pm 0.0096}$	**0.865**$_{\pm 0.0053}$	**0.636**$_{\pm 0.0092}$
Add FusA	2.222$_{\pm 0.0959}$	**0.795**$_{\pm 0.0181}$	0.951$_{\pm 0.0163}$	0.470$_{\pm 0.0230}$	0.861$_{\pm 0.0031}$	0.473$_{\pm 0.0033}$	0.703$_{\pm 0.0121}$	0.732$_{\pm 0.0048}$	0.863$_{\pm 0.0041}$	0.620$_{\pm 0.0031}$
Concat	2.213$_{\pm 0.1096}$	**0.795**$_{\pm 0.0195}$	0.949$_{\pm 0.0151}$	0.466$_{\pm 0.0225}$	0.859$_{\pm 0.0015}$	0.471$_{\pm 0.0062}$	0.700$_{\pm 0.0054}$	0.733$_{\pm 0.0049}$	0.863$_{\pm 0.0016}$	0.615$_{\pm 0.0038}$

Table E.3
Average results and standard deviation (subscripted) over 10 different seeds, for FusANet with different methods of initialization for global attention. Hybrid_H indicates Hybrid initialization using He Initialization, Hybrid_X indicates short for Hybrid initialization using He Initialization. We highlight the best result in bold.

	CP↓	BT↑	WC↑	LD↓	AD↑	CA↓	EY↑	HI↑	CH↑	GE↑
Uniform	2.220$_{\pm 0.1011}$	0.772$_{\pm 0.0213}$	0.948$_{\pm 0.0195}$	0.465$_{\pm 0.0217}$	0.864$_{\pm 0.0096}$	0.471$_{\pm 0.0132}$	0.711$_{\pm 0.0048}$	0.734$_{\pm 0.0232}$	0.860$_{\pm 0.0021}$	0.630$_{\pm 0.0051}$
Hybrid_H	2.192$_{\pm 0.0813}$	0.773$_{\pm 0.0231}$	0.950$_{\pm 0.0131}$	**0.467**$_{\pm 0.0441}$	0.865$_{\pm 0.0068}$	0.473$_{\pm 0.0084}$	0.710$_{\pm 0.0136}$	0.733$_{\pm 0.0041}$	0.862$_{\pm 0.0071}$	0.635$_{\pm 0.0080}$
Hybrid_X	**2.192**$_{\pm 0.0614}$	**0.779**$_{\pm 0.0158}$	**0.954**$_{\pm 0.0197}$	0.459$_{\pm 0.0354}$	**0.865**$_{\pm 0.0030}$	**0.466**$_{\pm 0.0041}$	**0.715**$_{\pm 0.0096}$	**0.735**$_{\pm 0.0053}$	**0.865**$_{\pm 0.0032}$	**0.636**$_{\pm 0.0092}$

References

Agarap, A. F. (2018). Deep learning using rectified linear units (reLU). https://api.semanticscholar.org/CorpusID:4090379. arXiv:1803.08375

Alvarez Melis, D., & Jaakkola, T. (2018). Towards robust interpretability with self-explaining neural networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 31*). https://proceedings.neurips.cc/paper_files/paper/2018/file/3e9f0fc9b2f89e043bc6233994dfcf76-Paper.pdf.

Arik, S. Ö., & Pfister, T. (2019). Tabnet: Attentive interpretable tabular learning. arXiv:1908.07442

Baldi, P., Sadowski, P., & Whiteson, D. (2014). Searching for exotic particles in high-energy physics with deep learning. *Nature Communications*, *5*.

Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M., & Kasneci, G. (2022). Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, (pp. 1–21). https://doi.org/10.1109/TNNLS.2022.3229161

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Bui, H., & Le, B. (2024). Ganet - learning tabular data using global attention. In N. T. Nguyen, B. Franczyk, A. Ludwig, M. Núñez, J. Treur, G. Vossen, & A. Kozierkiewicz (Eds.), *Computational collective intelligence* (pp. 95–106). Cham: Springer Nature Switzerland.

Chefer, H., Gur, S., & Wolf, L. (2021). Transformer interpretability beyond attention visualization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 782–791).

Chen, J., Liao, K., Wan, Y., Chen, D. Z., & Wu, J. (2022). Danets: Deep abstract networks for tabular data classification and regression. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3930–3938). (*vol. 36*).

Chen, J., Liao, K.-Y., Wan, Y., Chen, D. Z., & Wu, J. (2021). Danets: Deep abstract networks for tabular data classification and regression. arXiv:2112.02962

Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785–794).

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L. et al. (2020). Rethinking attention with performers. arXiv:2009.14794

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.

Fnu, N., & Bansal, A. (2024). Understanding the architecture of vision transformer and its variants: A review. In *2024 1st international conference on innovative engineering sciences and technological research (ICIESTR)* (pp. 1–6). https://doi.org/10.1109/ICIESTR60916.2024.10798341

Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *International conference on artificial intelligence and statistics*. https://api.semanticscholar.org/CorpusID:5575601.

Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. In *NeurIPS*.

Gorishniy, Y. V., Rubachev, I., Kartashev, N. K., Shlenskii, D., Kotelnikov, A., & Babenko, A. (2023). Tabr: Tabular deep learning meets nearest neighbors in 2023. https://api.semanticscholar.org/CorpusID:260164966.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Thirty-sixth conference on neural information processing systems datasets and benchmarks track*. https://openreview.net/forum?id=Fp7_phQszn.

Hao, Y., Dong, L., Wei, F., & Xu, K. (2021). Self-attention attribution: Interpreting information interactions inside transformer. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 12963–12971). (*vol. 35*).

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision* (pp. 1026–1034).

Hendrycks, D., & Gimpel, K. (2016). Gaussian error linear units (gelus). arXiv:1606.08415

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531

Huang, X., Khetan, A., Cvitkovic, M., & Karnin, Z. (2020). Tabtransformer: Tabular data modeling using contextual embeddings. arXiv:2012.06678

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2704–2713).

Kelley Pace, R., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, *33*(3), 291–297. https://www.sciencedirect.com/science/article/pii/S016771529600140X. https://doi.org/10.1016/S0167-7152(96)00140-X

Kohavi, R. (1996). Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid. In *Proceedings of the second international conference on knowledge discovery and data mining* KDD'96 (p. 202–207). AAAI Press.

Liu, Y., Li, H., Guo, Y., Kong, C., Li, J., & Wang, S. (2022). Rethinking attention-model explainability through faithfulness violation test. In *International conference on machine learning* (pp. 13807–13824). PMLR.

Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. arXiv:1711.05101

Louizos, C., Welling, M., & Kingma, D. P. (2017). Learning sparse neural networks through l_0 regularization. arXiv:1712.01312

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, *30*.

Madeo, R. C. B., Lima, C. A. M., & Peres, S. M. (2013). Gesture unit segmentation using support vector machines: Segmenting gestures from rest positions. In *Proceedings of the 28th annual ACM symposium on applied computing* SAC '13 (p. 46–52). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2480362.2480373

McDermott, J., & Forsyth, R. S. (2016). Diagnosing a disorder in a classification benchmark. *Pattern Recognition Letters*, *73*, 41–43.

Na, K., Lee, J.-H., & Kim, E. (2024). Lf-transformer: Latent factorizer transformer for tabular learning. *IEEE Access*, *12*, 10690–10698. https://api.semanticscholar.org/CorpusID:267157174.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). Catboost: Unbiased boosting with categorical features. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 31*). https://proceedings.neurips.cc/paper_files/paper/2018/file/14491b756b3a51daac41c24863285549-Paper.pdf.

Rohekar, R. Y., Gurwicz, Y., & Nisimov, S. (2023). Causal interpretation of self-attention in pre-trained transformers. *Advances in Neural Information Processing Systems*, *36*, 31450–31465.

Salojärvi, J., Puolamäki, K., Simola, J., Kovanen, L., Kojo, I., & Kaski, S. (2005). Inferring relevance from eye movements: Feature extraction.

Sanh, V., Wolf, T., & Rush, A. (2020). Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, *33*, 20378–20389.

Serrano, S., & Smith, N. A. (2019). Is attention interpretable? arXiv:1906.03731

Somepalli, G., Goldblum, M., Schwarzschild, A., Bruss, C. B., & Goldstein, T. (2021). Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. arXiv:2106.01342

Song, W., Shi, C., Xiao, Z., Duan, Z., Xu, Y., Zhang, M., & Tang, J. (2018). Autoint: Automatic feature interaction learning via self-attentive neural networks. *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Wang, Q., Li, B., Xiao, T., Zhu, J., Li, C., Wong, D. F., & Chao, L. S. (2019). Learning deep transformer models for machine translation. arXiv:1906.01787

Wang, S., Li, B. Z., Khabsa, M., Fang, H., & Ma, H. (2020). Linformer: Self-attention with linear complexity. arXiv:2006.04768

Wu, J., Chen, S., Zhao, Q., Sergazinov, R., Li, C., Liu, S., Zhao, C., Xie, T., Guo, H., Ji, C., Cociorva, D., & Brunzel, H. (2024). Switchtab: Switched autoencoders are effective tabular learners. arXiv:2401.02013 https://api.semanticscholar.org/CorpusID:266755624.

Yeh, I.-C., Yang, K.-J., & Ting, T.-M. (2009). Knowledge discovery on RFM model using bernoulli sequence. *Expert Systems with Applications*, *36*(3, Part 2), 5866–5871. https://www.sciencedirect.com/science/article/pii/S0957417408004508. https://doi.org/10.1016/j.eswa.2008.07.018